Lộ trình Backend Node.js

Dự án xuyên suốt **PingPulse**, làm theo quy trình của một thực tập sinh thật: ticket, nhánh, pull request, sprint 1 tuần, báo cáo hằng ngày, review code.

1. Tổng quan

Hạng mục	Nội dung
Điểm xuất phát	Biết JavaScript, Express, CRUD cơ bản với MongoDB. Chưa dùng TypeScript, chưa có nền SQL.
Mục tiêu 1	Có kỹ năng và sản phẩm để nộp CV intern/fresher trước Tết 2027 (khoảng 13 tuần từ 3/11/2026).
Mục tiêu 2	Đạt trình độ kỹ thuật Junior sau khoảng 6 tháng (khoảng tháng 5/2027).
Dự án	**PingPulse**: người dùng thêm URL cần theo dõi, hệ thống kiểm tra định kỳ, lưu lịch sử, cảnh báo khi sập, có trang trạng thái công khai.
Stack	TypeScript, Express (rồi NestJS), PostgreSQL + Prisma, Redis + BullMQ, Docker, GitHub Actions, Vitest/Jest.
Cách làm việc	Đóng vai thực tập sinh của "công ty PingPulse": mọi việc là ticket, mỗi ticket một nhánh và một pull request, mỗi tuần là một sprint.
Thời gian học	Giả định 20-25 giờ/tuần. Ít hơn thì cắt bớt tuần 12 (NestJS) và phần nâng cao.

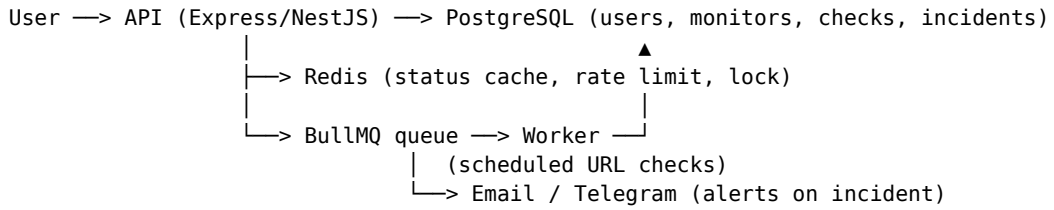
Khác gì so với bản PDF trước

- **Thêm mới:** Phần 2 (quy trình làm việc) và bảng **sprint backlog theo ticket** ở từng level.
- **Kiểm tra bài tự làm vs test chính thức:** từ tuần 1 bạn chỉ dùng `assert` hoặc `console.log` để tự kiểm tra hàm. Vitest/Jest, mock, integration test và CI vẫn học ở Level D (tuần 8-9) như lộ trình cũ.
- Nội dung kỹ thuật từng level giữ nguyên.
- **Lưu ý:** đây là mô phỏng. Nó giúp bạn quen quy trình nhưng không thay được thực tập thật (không có người thật review, không có áp lực production). Hãy vẫn nộp CV sớm.

Bản đồ 13 tuần

Tuần	Level	Sprint	PingPulse có thêm gì
1-2	A. TypeScript	S1, S2	Bộ hàm tiện ích + Checker Core (lõi kiểm tra URL)
3-4	B. Git, SQL	S3, S4	Data Layer: schema, 1 triệu dòng, truy vấn uptime
5-7	C. API + Prisma	S5-S7	PingPulse API: auth, monitor, lịch sử, chống SSRF, Swagger
8-9	D. Test, Docker, Deploy	S8, S9	Test, CI, chạy thật, tự giám sát chính nó
10-11	E. Redis, BullMQ	S10, S11	Engine tự động: lịch kiểm tra, cảnh báo, cache, rate limit
12	F. NestJS	S12	Refactor Auth + Monitor sang NestJS
13	G. CV	S13	Demo, video, README kiến trúc, CV
Tháng 2-5	H. Junior	S14+	Đồng thời, partition, Nginx, multi-region, system design

Kiến trúc đích



2. Cách vận hành: công ty PingPulse

Phần này là "luật chơi" áp dụng cho mọi level. Tuần đầu chỉ cần làm đúng bốn thứ: **ticket, nhánh, pull request, devlog**. Các thứ còn lại thêm dần. Quy trình chỉ nên chiếm khoảng 10-15% thời gian, phần còn lại là viết code.

2.1 Ai đóng vai gì

Vai	Ai đóng	Việc
Developer (intern)	Bạn	Nhận ticket, code, mở PR, trả lời review
Product Owner / khách hàng	Bạn tự viết ticket; nhờ AI thỉnh thoảng đổi yêu cầu	Quyết định làm gì trước, viết tiêu chí hoàn thành
Mentor / reviewer	AI (chỉ review và gợi ý, không viết hộ code), cộng đồng dev nếu có	Chỉ ra lỗi, hỏi "tại sao bạn viết thế này"
Người dùng thật	Bạn bè dùng thử PingPulse	Báo lỗi, góp ý

2.2 Công cụ (bản miễn phí, tương đương công ty)

Công ty thường dùng	Bạn dùng	Ghi chú
Jira	GitHub Issues + GitHub Projects (bảng Kanban)	Quen Jira sau cũng nhanh vì khái niệm giống nhau; có thể thử Jira miễn phí
Slack	File devlog.md (báo cáo hằng ngày) và Discord/Zalo nhóm bạn nếu có	Tập viết tin nhắn rõ ràng
Google Drive	Thư mục docs/ trong repo	Tài liệu thiết kế, quyết định kỹ thuật, post-mortem
Git	Git + GitHub	Nhánh và PR cho mọi thay đổi

2.3 Nhịp một sprint (1 tuần)

Ngày	Hoạt động	Sản phẩm
Thứ 2 sáng	**Sprint Planning**: chọn ticket từ backlog, ước lượng giờ (tối đa 20-25 giờ)	Bảng sprint trên GitHub Projects
Mỗi ngày	**Daily**: 3 dòng vào devlog.md (hôm qua, hôm nay, vướng gì)	devlog.md
Thứ 2-6	Làm ticket: nhánh → commit → PR → review → merge	PR đã merge
Thứ 6 chiều	**Sprint Review**: chạy thử, quay màn hình 1-2 phút những gì đã xong	Video hoặc ảnh chụp trong README
Thứ 6 tối	**Retrospective**: viết 3 dòng (tốt, chưa tốt, tuần sau đổi gì) và so giờ ước lượng với giờ thực tế	Mục retro trong devlog.md

2.4 Mẫu ticket (GitHub Issue)

```
Tiêu đề: [PP-2] Viết hàm chunk
Là: developer của PingPulse
Muốn: có hàm chia mảng thành các nhóm có kích thước cố định
Để: dùng lại khi xử lý danh sách monitor theo lô

Tiêu chí hoàn thành:
- chunk([1,2,3,4,5], 2) trả [[1,2],[3,4],[5]]
- Mảng rỗng trả []
- size <= 0 ném lỗi có thông báo rõ ràng
- tsc --noEmit sạch lỗi

Ước lượng: 1 giờ     Nhãn: feat, level-a, priority-medium
```

2.5 Quy ước Git

Hạng mục	Quy ước	Ví dụ
Nhánh	**loại/số-ticket-mô-tả-ngắn**; loại: feat, fix, docs, chore, refactor, test	`feat/2-chunk-util`
Commit	Dạng "loại: mô tả", thể mệnh lệnh, mỗi commit một việc	`fix: throw when size is not positive`
Nhánh chính	`main` luôn chạy được. Không code thẳng vào main	Chỉ merge qua pull request
Pull request	Một PR một ticket, đủ nhỏ để review trong 10-15 phút	Tiêu đề trùng tiêu đề ticket
Merge	Squash merge để lịch sử main gọn	Xóa nhánh sau khi merge

Đây là quy ước phổ biến. Khi đi thực tập thật, hãy làm theo quy ước của công ty đó.

2.6 Mẫu Pull Request

```
## Làm gì
Thêm hàm chunk vào src/utils.ts (PP-2)

## Vì sao
Cần chia danh sách monitor thành lô để kiểm tra theo đợt

## Cách kiểm tra
- npx tsx src/utils.check.ts
- npx tsc --noEmit

## Lưu ý cho reviewer
Mình chưa chắc nên ném lỗi hay trả mảng rỗng khi size <= 0, đã chọn ném lỗi vì ...

## Checklist
[ ] Đạt Definition of Done   [ ] Đã tự đọc lại diff   [ ] Đã cập nhật README nếu cần
```

2.7 Definition of Done (áp dụng cho mọi ticket)

- Đạt đủ tiêu chí hoàn thành trong ticket.
- `npx tsc --noEmit` và lint không báo lỗi.
- Đã kiểm tra các trường hợp biên (rỗng, null, số âm, dữ liệu sai).
- Từ Level D trở đi: có test và CI xanh.
- Đã mở PR, được review (AI hoặc người) và bạn đã trả lời hết comment.
- Đã cập nhật README hoặc tài liệu liên quan nếu hành vi thay đổi.
- Đã merge vào main và đóng ticket.

2.8 Báo cáo hằng ngày (devlog.md)

```
## 2026-11-04 (Sprint 1, ngày 2)
Hôm qua: dựng repo, bật strict, xong chunk và groupBy
Hôm nay: pick, omit, mở PR cho PP-2 và PP-3
Vướng: pick báo lỗi "Type K cannot be used to index type T", đã thử extends keyof T, chưa hiểu vì sao Pick<T, K> cần ràng buộc
Giờ thực tế: 4 giờ (ước lượng 3 giờ)
```

2.9 Cách đặt câu hỏi tốt

- **Mục tiêu:** bạn đang cố làm gì.
- **Đã thử:** bạn đã làm gì và kết quả ra sao.
- **Lỗi nguyên văn:** dán thông báo lỗi, không mô tả chung chung.
- **Code tối thiểu** tái hiện được vấn đề.
- **Câu hỏi rõ ràng:** cần gợi ý hướng đi hay cần xem code.

Mẹo: nhiều lần bạn tự tìm ra đáp án ngay trong lúc viết câu hỏi. Quy tắc 30 phút: kẹt quá 30 phút sau khi đã tự thử thì hỏi, đừng hỏi ngay từ phút thứ 5.

2.10 Clean code và checklist review

Clean code	Khi review PR, kiểm tra
Tên biến, hàm nói được việc nó làm (getActiveMonitors tốt hơn getData)	PR có đúng một mục đích không, có đủ nhỏ không
Hàm ngắn, làm một việc; quá khoảng 30-40 dòng thì cân nhắc tách	Có đạt tiêu chí của ticket không
Không lặp code, tách thành hàm dùng lại	Có trường hợp biên nào bị bỏ sót, có lỗi bảo mật không (validate, SSRF, SQL injection)
Comment giải thích "vì sao", không lặp lại "cái gì"	Có chỗ nào dùng any hoặc as mà không có lý do
Theo convention (ESLint + Prettier từ tuần 1)	Tên, cấu trúc thư mục, commit message có theo quy ước không

2.11 Tài liệu

- **README:** mô tả, cách chạy trong 3 lệnh, sơ đồ kiến trúc, danh sách chức năng. Cập nhật cuối mỗi level.
- **API doc:** Swagger/OpenAPI từ Level C.
- **Quyết định kỹ thuật:** mỗi lần chọn công nghệ, viết 5-10 dòng vào `docs/decisions/`: bối cảnh, lựa chọn, phương án đã bỏ, vì sao.
- **Post-mortem:** mỗi sự cố, viết vào `docs/postmortems/`: chuyện gì xảy ra, nguyên nhân gốc, cách phòng.

2.12 Sự kiện giả lập (tự tạo để giống đời thực)

Khi nào	Sự kiện	Bạn phải làm
Tuần 3	Khách đổi yêu cầu: "cần thêm kiểm tra theo method POST"	Đánh giá ảnh hưởng, báo ước lượng mới, cập nhật ticket
Tuần 6	Bạn bè báo bug khi đăng nhập	Tạo bug ticket, tái hiện, sửa, ghi nguyên nhân gốc
Tuần 9	Deploy lần đầu bị lỗi (chắc chắn xảy ra)	Viết post-mortem
Tuần 11	Khách muốn thêm kênh cảnh báo Telegram	Ước lượng, ưu tiên so với ticket đang làm
Tuần 12	Yêu cầu chuyển sang framework mới	Viết tài liệu quyết định kỹ thuật

Khi nào	Sự kiện	Bạn phải làm
Tháng 3	Dữ liệu phình to, trang thống kê chậm	Đo, tối ưu, báo cáo số liệu

3. Level A – TypeScript (Tuần 1-2, Sprint 1-2)

Mục tiêu: viết thành thạo TypeScript strict. **Sản phẩm:** bộ hàm tiện ích và Checker Core.

Nội dung học

Tuần	Chủ đề
1	Kiểu cơ bản, interface vs type, union, optional, any vs unknown, enum vs string union, hàm có kiểu, narrowing, tsconfig strict, noUncheckedIndexedAccess
2	Generics + constraint, utility types, discriminated union, type guard, as const/satisfies/readonly/import type, Promise/Awaited/ReturnType, Zod + z.infer, error handling, fetch + AbortSignal.timeout

Dựng dự án

```
npm init -y   (thêm "type": "module" vào package.json)
npm i -D typescript tsx @types/node eslint prettier
npx tsc --init   # strict, noUncheckedIndexedAccess, module/moduleResolution NodeNext, noEmit
# import tương đ□i ph□i có đuôi .js, ví dụ: import { chunk } from "./utils.js"
```

Sprint 1 – Bộ hàm tiện ích (tuần 1)

Mục tiêu sprint: 10 hàm tiện ích có kiểu đầy đủ, strict, không dùng any.

Ticket	Việc cần làm	Ước lượng
PP-1	Dựng repo, tsconfig strict, ESLint + Prettier, CONTRIBUTING.md (quy ước nhánh, commit)	2 giờ
PP-2	Viết chunk, groupBy, pick	3 giờ
PP-3	Viết omit, unique, compact (type guard x is T)	3 giờ
PP-4	Viết sleep, debounce (generic giữ nguyên tham số hàm gốc)	3 giờ
PP-5	Viết retry (async, backoff) và safeJsonParse (trả Result, data là unknown)	4 giờ
PP-6	Nhờ AI review toàn bộ, sửa theo nhận xét, viết retro	2 giờ

✎ BÀI THỰC HÀNH

1. Viết 10 hàm: `chunk, groupBy, pick, omit, unique, compact, sleep, debounce, retry, safeJsonParse`. Viết chữ ký trước, thân hàm sau.
2. Với mỗi hàm, tự kiểm tra bằng `node:assert` hoặc console.log, gồm cả trường hợp biên (mảng rỗng, size bằng 0 hoặc âm, JSON hỏng). Đây là kiểm tra cá nhân, chưa phải test chính thức.
3. Kiểm tra kiểu bằng `@ts-expect-error`: dòng sai kiểu phải thật sự báo lỗi (ví dụ truy cập key không có trong kết quả pick).
4. Tạo kiểu `Result<T, E>` bằng discriminated union và dùng nó trong safeJsonParse.
5. Tạo class `AppError` có statusCode và hàm `handleError(error: unknown)` xử lý an toàn.

★ MINI PROJECT: Bộ hàm tiện ích (PingPulse level 1a)

• Mỗi hàm một ticket nhỏ, một nhánh, một PR. Cuối tuần repo có khoảng 10 PR đã merge.

- **Quy tắc:** không any; as chỉ dùng khi có comment giải thích; tsc --noEmit sạch lỗi.
- devlog.md có đủ 5 ngày, retro ghi rõ giờ ước lượng và giờ thực tế.

Sprint 2 – Checker Core (tuần 2)

Mục tiêu sprint: hàm kiểm tra sức khỏe một website, chạy được từ CLI và qua một endpoint.

Ticket	Việc cần làm	Ước lượng
PP-7	Định nghĩa Result và CheckError (timeout, dns_error, ssl_error, connection_refused, bad_status) bằng discriminated union	2 giờ
PP-8	Viết checkUrl dùng fetch + AbortSignal.timeout, đo thời gian bằng performance.now()	4 giờ
PP-9	Phân loại lỗi mạng đúng (TypeError fetch failed, đọc error.cause.code)	3 giờ
PP-10	Validate cấu hình bằng Zod, viết CLI đọc urls.json và in bảng kết quả	4 giờ
PP-11	Endpoint POST /check bằng Express, middleware validate(schema), error handler tập trung	4 giờ
PP-12	README (cách chạy, ví dụ kết quả), review bằng AI, retro	2 giờ

★ MINI PROJECT: Checker Core (PingPulse level 1b)

- `checkUrl(url, options): Promise<Result<CheckResult, CheckError>>` trả status code, thời gian phản hồi, kích thước body.
- Mỗi loại lỗi phải được phân loại đúng: hãy thử với URL không tồn tại, cổng đóng, chứng chỉ hết hạn (badssl.com), trả về 500, và rất chậm.
- Cấu trúc: src/checker, src/routes, src/schemas, src/middlewares. Lỗi trả JSON thống nhất.

✓ HOÀN THÀNH KHI

- `tsc --noEmit` sạch, không còn any vô lý do.
- Giải thích được unknown khác any, vì sao dùng union mô tả lỗi tốt hơn ném exception tùy tiện.
- Có ít nhất 12 PR đã merge và 10 ngày devlog.

4. Level B – Git, Linux, SQL/PostgreSQL (Tuần 3-4, Sprint 3-4)

Mục tiêu: SQL ở mức đủ phỏng vấn. **Sản phẩm:** Data Layer của PingPulse với 1 triệu dòng lịch sử.

Nội dung học

Tuần	Chủ đề
3	Git nâng cao (rebase, conflict, PR), Linux cơ bản (`ls, grep, curl, ssh, chmod, tail -f`), SQL: SELECT, WHERE, GROUP BY, JOIN, khóa chính/ngoại, chuẩn hóa
4	Index (B-tree, composite), `EXPLAIN ANALYZE`, transaction và ACID, N+1, CTE và window function, SQL vs MongoDB

```
docker run --name pg -e POSTGRES_PASSWORD=secret -p 5432:5432 -d postgres:16
```

Sprint 3 – Nền tảng SQL (tuần 3)

Sự kiện giả lập: khách đổi yêu cầu, thêm kiểm tra bằng method POST. Hãy cập nhật ticket và báo ước lượng.

Ticket	Việc cần làm	Ước lượng
PP-13	Thực hành Git: tạo conflict có chủ đích, giải quyết, rebase -i gộp commit	2 giờ
PP-14	30 bài SQL (SQLBolt, pgexercises)	6 giờ
PP-15	Script shell sao lưu DB bằng pg_dump ra file có timestamp	2 giờ
PP-16	Vẽ ERD 5 bảng: users, monitors, checks, incidents, status_pages (dbdiagram.io)	3 giờ
PP-17	Cập nhật Checker Core: thêm option method (GET, POST, HEAD)	2 giờ

Sprint 4 – Data Layer (tuần 4)

Ticket	Việc cần làm	Ước lượng
PP-18	Viết schema.sql có khóa ngoại, ràng buộc, index cơ bản	3 giờ
PP-19	Sinh 100 monitor và 1 triệu dòng checks (generate_series hoặc script TS)	3 giờ
PP-20	Truy vấn: uptime % 30 ngày, p95 thời gian phản hồi, sự cố 7 ngày, top 10 monitor chậm	5 giờ
PP-21	EXPLAIN ANALYZE, thêm composite index (monitor_id, checked_at), ghi REPORT.md trước/sau	4 giờ
PP-22	Transaction mở sự cố (đóng incident cũ, tạo mới, cập nhật trạng thái), thử rollback	3 giờ

✎ BÀI THỰC HÀNH

1. Làm 30 bài SQL trên SQLBolt, pgexercises.com hoặc LeetCode Database.
2. Tạo bảng 1 triệu dòng bằng `generate_series`, đo thời gian truy vấn trước/sau khi thêm index.
3. Mô phỏng chuyển tiền trong transaction; cố tình lỗi giữa chừng và ROLLBACK.
4. Tạo tình huống N+1 trong Node.js rồi sửa bằng JOIN hoặc IN.

★ MINI PROJECT: Data Layer (PingPulse level 2)

- `schema.sql`, seed 1 triệu dòng, 4-5 truy vấn nghiệp vụ, `REPORT.md` có số liệu trước/sau khi thêm index.
- Transaction mở/đóng sự cố có thử rollback.

✓ HOÀN THÀNH KHI

- Giải thích được 3 dạng JOIN, cách index hoạt động, vì sao thứ tự cột trong composite index quan trọng.
- Nói được khi nào chọn PostgreSQL, khi nào chọn MongoDB.

5. Level C – API + Prisma (Tuần 5-7, Sprint 5-7)

Mục tiêu: backend hoàn chỉnh về chức năng, kiến trúc sạch, bảo mật cơ bản. **Sản phẩm:** PingPulse API.

Nội dung học

Tuần	Chủ đề
5	Prisma (schema, migration, relation, transaction, tránh N+1), Controller - Service - Repository, pagination offset và cursor
6	Hash password (argon2/bcrypt), JWT access + refresh token (rotate, revoke), phân quyền role
7	OWASP Top 10, CORS, helmet, SQL injection, SSRF (rất liên quan vì app gọi URL do người dùng nhập), Swagger, pino, graceful shutdown

Sprint 5 – Prisma và kiến trúc (tuần 5)

Ticket	Việc cần làm	Ước lượng
PP-23	Prisma schema, migration, seed từ Data Layer	4 giờ
PP-24	CRUD monitor theo Controller - Service - Repository	6 giờ
PP-25	Phân trang cursor cho danh sách; so sánh với offset trên bảng 1 triệu dòng	4 giờ
PP-26	Bật query log, kiểm tra và sửa N+1	2 giờ

Sprint 6 – Xác thực và phân quyền (tuần 6)

Sự kiện giả lập: bạn bè báo bug khi đăng nhập. Tạo bug ticket, tái hiện, sửa, ghi nguyên nhân gốc.

Ticket	Việc cần làm	Ước lượng
PP-27	Đăng ký, đăng nhập, hash password bằng argon2	4 giờ
PP-28	JWT access + refresh token rotation, thu hồi phiên khi token cũ bị dùng lại, logout	6 giờ
PP-29	Middleware auth và requireRole (user, admin)	3 giờ
PP-30	Giới hạn gói free: tối đa 5 monitor mỗi user	2 giờ

Sprint 7 – Chức năng chính và bảo mật (tuần 7)

Ticket	Việc cần làm	Ước lượng
PP-31	POST /monitors/:id/check gọi Checker Core và lưu vào bảng checks	4 giờ

Ticket	Việc cần làm	Ước lượng
PP-32	GET /monitors/:id/checks (cursor) và /stats (uptime %, p95)	4 giờ
PP-33	Chống SSRF: chặn URL trỏ tới IP nội bộ, kể cả sau khi redirect	4 giờ
PP-34	Swagger, pino logging, helmet, CORS whitelist, /health, xử lý SIGTERM	5 giờ

✎ BÀI THỰC HÀNH

1. Viết middleware `auth` và `requireRole`; thử tấn công bằng token hết hạn và token giả mạo.
2. Cài refresh token rotation; token cũ bị dùng lại thì thu hồi toàn bộ phiên.
3. Thử SQL injection trên endpoint dùng raw query rồi sửa bằng tham số hóa.
4. Thử gọi PingPulse với URL http://127.0.0.1 và địa chỉ metadata cloud, xác nhận bị chặn.

★ MINI PROJECT: PingPulse API (PingPulse level 3)

- Khoảng 15-20 endpoint chạy ổn: auth, monitor, check now, lịch sử, thống kê, quản trị.
- Swagger tại /docs, README có ERD và danh sách quyết định thiết kế.

✔ HOÀN THÀNH KHI

- Mọi endpoint đều có validate và phân quyền.
- Giải thích được flow refresh token, SSRF là gì và bạn chặn nó như thế nào.

6. Level D – Test, Docker, CI/CD, Deploy (Tuần 8-9, Sprint 8-9)

Mục tiêu: PingPulse ở mức có thể nộp cho nhà tuyển dụng. **Bắt đầu nộp CV từ tuần này.** Đây là nơi học **test chính thức** (Vitest/Jest), khác với việc tự kiểm tra bằng assert ở các tuần trước.

Tuần	Chủ đề
8	Unit test cho Service (mock Repository), integration test bằng supertest + DB test, mô hình AAA, coverage, mock HTTP bên ngoài (MSW/nock)
9	Dockerfile multi-stage, docker-compose (app + PostgreSQL + Redis), GitHub Actions (lint, test, build), deploy VPS/Railway/Render, Nginx, HTTPS

Sprint 8 – Test (tuần 8)

Ticket	Việc cần làm	Ước lượng
PP-35	Cài Vitest/Jest; chuyển các đoạn assert cũ thành test chính thức	3 giờ
PP-36	Logic sự cố: fail 3 lần liên tiếp mở incident, thành công 2 lần liên tiếp đóng incident, kèm unit test kỹ	5 giờ
PP-37	Unit test cho Auth và Monitor Service (cả trường hợp lỗi)	5 giờ
PP-38	Integration test: đăng ký, đăng nhập, tạo monitor, check now, xem stats; mock website bên ngoài	6 giờ

Sprint 9 – Đóng gói và deploy (tuần 9)

Sự kiện giả lập: lần deploy đầu tiên lỗi. Viết post-mortem vào docs/postmortems/.

Ticket	Việc cần làm	Ước lượng
PP-39	Dockerfile multi-stage; so sánh dung lượng image trước/sau	3 giờ
PP-40	docker-compose (app + Postgres + Redis) có healthcheck	3 giờ
PP-41	GitHub Actions: lint, test với service Postgres, build image; badge trong README	4 giờ
PP-42	Deploy lên VPS hoặc dịch vụ miễn phí, HTTPS, biến môi trường, .env.example	6 giờ
PP-43	Dogfooding: tạo monitor trên chính PingPulse để theo dõi /health của nó	1 giờ

Mẫu Dockerfile multi-stage

```
FROM node:22-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npx prisma generate && npm run build

FROM node:22-alpine
WORKDIR /app
ENV NODE_ENV=production
COPY --from=build /app/dist ./dist
COPY --from=build /app/node_modules ./node_modules
COPY package*.json ./
USER node
CMD ["node", "dist/main.js"]
```

✎ BÀI THỰC HÀNH

1. Viết 15 unit test cho Service, gồm cả trường hợp lỗi (sai mật khẩu, vượt giới hạn gói free).
2. Viết docker-compose chạy app + Postgres bằng một lệnh, dùng healthcheck và depends_on.
3. Tạo workflow GitHub Actions chạy lint, test, build mỗi pull request.

★ MINI PROJECT: Ship it + Dogfooding (PingPulse level 4)

- Coverage phần Service từ 70%, CI xanh, có link chạy thật với HTTPS.
- Từ giờ, ticket nào cũng phải có test mới được merge (cập nhật Definition of Done).

✓ HOÀN THÀNH KHI

- Giải thích được multi-stage build để làm gì, vì sao integration test cần DB thật, vì sao phải mock mạng khi test.
- Đã có link demo và sẵn sàng nộp CV.

7. Level E – Redis, Rate Limiter, BullMQ (Tuần 10-11, Sprint 10-11)

Mục tiêu: PingPulse tự chạy 24/7 và biết báo động. **Sản phẩm:** Engine tự động.

Tuần	Chủ đề
10	Redis (string, hash, set, sorted set, TTL), cache-aside, invalidation, cache stampede, rate limiter (fixed rồi sliding window)
11	BullMQ: producer/consumer, repeatable jobs, retry và backoff, concurrency, dead-letter, idempotency, bull-board

Sprint 10 – Cache và giới hạn request (tuần 10)

Ticket	Việc cần làm	Ước lượng
PP-44	Thực hành redis-cli: SET, EXPIRE, INCR, ZADD, ZREMRANGEBYSCORE	2 giờ
PP-45	GET /status/:slug công khai, cache-aside bằng Redis (TTL 30 giây), xóa cache khi dữ liệu đổi	5 giờ
PP-46	Rate limiter fixed window, nâng cấp thành sliding window; header X-RateLimit-Remaining, Retry-After	6 giờ
PP-47	Load test trang status bằng autocannon trước/sau khi có cache, ghi số liệu vào README	3 giờ

Sprint 11 – Hàng đợi và cảnh báo (tuần 11)

Sự kiện giả lập: khách muốn thêm kênh cảnh báo Telegram. Ước lượng và so với ticket đang làm trước khi nhận.

Ticket	Việc cần làm	Ước lượng
PP-48	BullMQ repeatable jobs: mỗi monitor một lịch theo interval; tạo/sửa/xóa monitor thì cập nhật lịch	6 giờ
PP-49	Worker chạy trong container riêng, giới hạn concurrency, retry và backoff	4 giờ
PP-50	Cảnh báo email/Telegram khi mở/đóng incident qua queue, idempotency key chống gửi trùng	6 giờ
PP-51	bull-board, xử lý job thất bại, README mục "cách vận hành"	3 giờ

✎ BÀI THỰC HÀNH

1. Đo thời gian phản hồi trước/sau cache bằng autocannon.
2. Viết job gửi email giả lập với retry 3 lần, backoff theo cấp số nhân.
3. Cố tình làm worker lỗi, quan sát job thất bại và viết cách xử lý lại.

★ MINI PROJECT: Engine tự động (PingPulse level 5)

- PingPulse tự kiểm tra theo lịch, tự mở/đóng sự cố, tự gửi cảnh báo, không gửi trùng khi retry.
- Thử tắt một website mẫu và xác nhận bạn nhận được cảnh báo.

✔ HOÀN THÀNH KHI

• Giải thích được cache-aside, sliding window khác fixed window thế nào, vì sao cần idempotency khi có retry.

8. Level F – NestJS (Tuần 12, Sprint 12)

Mục tiêu: làm quen NestJS (nhiều công ty tại Việt Nam dùng). **Sự kiện giả lập:** khách yêu cầu chuyển sang framework mới, hãy viết tài liệu quyết định kỹ thuật.

Ticket	Việc cần làm	Ước lượng
PP-52	Làm theo tài liệu NestJS: Module, Controller, Provider, DI qua ứng dụng Todo	5 giờ
PP-53	Viết ADR (docs/decisions): vì sao chuyển sang NestJS, phương án đã bỏ	1 giờ
PP-54	Chuyển module Auth: DTO + class-validator, JwtAuthGuard, decorator @Roles	6 giờ
PP-55	Chuyển module Monitor; chạy lại bộ integration test cũ để chứng minh không hỏng	6 giờ
PP-56	README mục "Express vs NestJS: mình rút ra gì"	1 giờ

✔ HOÀN THÀNH KHI

• Giải thích được Dependency Injection và lợi ích khi viết test.

9. Level G – Demo và CV (Tuần 13, Sprint 13)

Ticket	Việc cần làm	Ước lượng
PP-57	Status page công khai cho chính PingPulse, để chạy liên tục	2 giờ
PP-58	Quay video demo 2 phút: thêm monitor, tắt website mẫu, nhận cảnh báo	3 giờ
PP-59	Sơ đồ kiến trúc và mục "Quyết định thiết kế" trong README	3 giờ
PP-60	CV 1 trang: PingPulse với 3 gạch đầu dòng có số liệu (ví dụ p95 giảm từ A xuống B nhờ cache)	3 giờ
PP-61	Dọn GitHub: pin repo, README đẹp, xóa repo rác	1 giờ
PP-62	Mock interview 2-3 buổi, trả lời thành tiếng 30 câu hỏi (bảng dưới), 20 bài LeetCode Easy	8 giờ

Câu hỏi phỏng vấn thường gặp

Chủ đề	Câu hỏi
JS/Node	Event Loop hoạt động ra sao? `process.nextTick` khác `setImmediate`? Promise vs async/await? Vì sao Node mạnh về I/O nhưng yếu với tác vụ CPU nặng?
TypeScript	any vs unknown? interface vs type? Generic dùng để làm gì? Discriminated union là gì?
SQL	Các loại JOIN? Index là gì, khi nào không nên đánh index? ACID? N+1 là gì? SQL vs NoSQL?
API và bảo mật	PUT vs PATCH? 401 vs 403? Cursor vs offset? Idempotent là gì? JWT, refresh token, SSRF, SQL injection, XSS, CSRF là gì?
Hạ tầng	Docker image vs container? Multi-stage build? Redis dùng để làm gì? Khi nào cần message queue?

Chủ đề	Câu hỏi
Về PingPulse	Nếu có 100.000 monitor thì nghẽn ở đâu? Làm sao tránh một monitor bị kiểm tra trùng? Làm sao tránh gửi cảnh báo trùng? Làm sao chặn SSRF?
Quy trình	Bạn đã làm việc theo Scrum chưa? Bạn ước lượng công việc thế nào? Khi bị kẹt bạn làm gì? Bạn review code ra sao? (Hãy trả lời bằng chính trải nghiệm trong PingPulse.)

10. Level H – Lên Junior (Tháng 2-5/2027, Sprint 14 trở đi)

Mục tiêu: giải quyết bài toán đồng thời và dữ liệu lớn, hiểu system design ở mức Junior. Hiệu quả nhất khi bạn **vừa thực tập/làm fresher vừa học**. Sprint có thể kéo dài 2 tuần ở giai đoạn này vì mỗi ticket lớn hơn.

H1. Đồng thời: chống kiểm tra trùng (Tháng 2-3)

Ticket	Việc cần làm	Ước lượng
PP-63	Chạy 3 worker, viết script đếm số lần một monitor bị kiểm tra trùng trong 1 phút (chứng minh lỗi tồn tại)	4 giờ
PP-64	Sửa bằng 3 cách: Redis lock (SET NX PX), SELECT FOR UPDATE SKIP LOCKED, Redis Lua Script; so sánh hiệu năng	10 giờ
PP-65	Dead-letter queue cho lỗi hệ thống (khác với website sập), sliding window rate limiter hoàn chỉnh	6 giờ
PP-66	Load test bằng k6, chứng minh không kiểm tra trùng, đính kèm biểu đồ	5 giờ

★ MINI PROJECT: Distributed Scheduler (PingPulse level 8)

- Nhiều worker chạy song song nhưng mỗi monitor chỉ được kiểm tra đúng một lần mỗi chu kỳ; có số liệu chứng minh.

H2. Dữ liệu lớn: partition và rollup (Tháng 3-4)

Sự kiện giả lập: dữ liệu phình to, trang thống kê chậm. Đo, tối ưu, báo cáo số liệu.

Ticket	Việc cần làm	Ước lượng
PP-67	Tái hiện non-repeatable read và phantom read trên 2 phiên psql, sửa bằng isolation level phù hợp	4 giờ
PP-68	Partition bảng checks theo tháng, job dọn dữ liệu cũ hơn 90 ngày	8 giờ
PP-69	Bảng checks_daily tổng hợp uptime/p95 theo ngày bằng job nền; trang thống kê đọc từ đó	8 giờ
PP-70	Báo cáo: thời gian truy vấn trước/sau, ảnh hưởng tới tốc độ ghi	3 giờ

★ MINI PROJECT: Data at Scale (PingPulse level 9)

- Bảng 20 triệu dòng truy vấn nhanh nhờ partition và rollup, có báo cáo số liệu.

H3. Vận hành và system design (Tháng 4-5)

Ticket	Việc cần làm	Ước lượng
PP-71	Nginx cân bằng tải 2 instance, tắt một instance và xác nhận hệ thống vẫn phục vụ	6 giờ
PP-72	Sentry, log có correlation ID, graceful shutdown hoàn chỉnh	5 giờ
PP-73	Kiểm tra từ 2 vùng (2 VPS) để tránh báo sai khi chỉ một vùng mất kết nối	10 giờ
PP-74	Tài liệu thiết kế 1-2 trang: mở rộng PingPulse lên 100.000 monitor (nút thắt, giải pháp, đánh đổi)	6 giờ
PP-75	Thiết kế trên giấy: URL shortener, hệ thống thông báo; review 5 PR của người khác	8 giờ

★ MINI PROJECT: Production Hardening (PingPulse level 10)

- PingPulse chạy ổn định nhiều tuần, có báo cáo p95 và throughput trước/sau tối ưu.

✔ HOÀN THÀNH KHI

- Giải quyết được bài toán đồng thời và chứng minh bằng load test.
- Trình bày được thiết kế hệ thống đơn giản cùng các đánh đổi.
- PingPulse chạy ổn định vài tuần; bạn đã đi phỏng vấn nhiều vòng.

Để sau mốc Junior

- Microservices, gRPC, API Gateway, saga/outbox, circuit breaker.
- Observability (Prometheus, Grafana, tracing), Kubernetes, performance profiling, security audit.
- Hệ thống chạy code người dùng bằng dockerode: hay nhưng rủi ro bảo mật cao, chỉ làm khi đã có kinh nghiệm.

11. Phụ lục: mẫu dùng ngay và tài nguyên

Mẫu CONTRIBUTING.md

```
# Quy ước làm việc
Nhánh: loại/số-ticket-mô-tả (feat/2-chunk-util)
Commit: loại: mô tả ngắn, thể mệnh lệnh (fix: handle empty array)
Không code thẳng vào main. Mỗi ticket một PR, squash merge.
Một PR chỉ làm một việc và đủ nhỏ để review trong 10-15 phút.

# Definition of Done
- Đạt tiêu chí của ticket, tsc và lint sạch
- Đã kiểm tra trường hợp biên (có test từ Level D)
- PR đã được review và trả lời hết comment
- Đã cập nhật tài liệu nếu cần
```

Mẫu retro cuối sprint

```
## Retro Sprint N
Đã xong: PP-2, PP-3 ...   Chưa xong: PP-5 (vì ...)
Giờ ước lượng: 20   Giờ thực tế: 26   Lệch vì: ...
Tốt: ...
Chưa tốt: ...
Tuần sau đổi: ...
```

Mẫu post-mortem

```
## Sự cố: [tên ngắn]   Ngày: ...
Chuyện gì xảy ra: ...
Ảnh hưởng: ...
Nguyên nhân gốc: ...
Đã xử lý thế nào: ...
Cách phòng lần sau: ...
```

Lịch học mỗi tuần

Ngày	Hoạt động
Thứ 2	Sprint Planning, chọn ticket, bắt đầu học chủ đề của tuần
Thứ 3-5	Học và làm ticket (2-4 giờ/ngày), daily vào devlog.md, mở PR
Thứ 6	Hoàn tất PR, Sprint Review (quay màn hình), Retrospective
Thứ 7	Việc tồn đọng, đọc thêm, tập trả lời câu hỏi phỏng vấn
Chủ nhật	Nghỉ

Linux: chỉ cần mức cơ bản

- File: `cd, ls, cp, mv, rm, mkdir, cat, nano`. Log: `tail -f, grep, less`. Quyền: `chmod, chown`.
- Mạng: `ssh, curl, scp, ss -tulpn`. Tiến trình: `ps, kill, systemctl`.
- Người dùng Windows: cài WSL2 (Ubuntu), code và chạy Docker trong đó.

Tài nguyên

Chủ đề	Nguồn gợi ý
TypeScript	TypeScript Handbook (typescriptlang.org), Total TypeScript của Matt Pocock
Node.js	nodejs.org, roadmap.sh/nodejs
SQL/PostgreSQL	SQLBolt, pgexercises.com, tài liệu PostgreSQL, use-the-index-luke.com
Prisma / NestJS	prisma.io và docs.nestjs.com
Redis / BullMQ	redis.io (Redis University miễn phí), docs.bullmq.io
System Design	Sách "Designing Data-Intensive Applications" (đọc dần), kênh ByteByteGo
Bảo mật	OWASP Top 10 và mục SSRF (owasp.org)
Agile/Scrum	Scrum Guide (scrumguides.org, bản tiếng Việt có sẵn)

Lưu ý cuối

- Đây là bản **mô phỏng** thực tập. Hãy tìm thêm feedback từ người thật: cộng đồng dev, bạn bè dùng thử PingPulse, issue "good first issue" trên GitHub.
- Nếu trễ tiến độ, hãy **cắt phạm vi** (bỏ NestJS, bỏ phần nâng cao của level H) chứ đừng cắt test, deploy, README và quy trình PR.
- "Trình độ kỹ thuật Junior" bạn tự đạt được, còn "được công nhận Junior" phụ thuộc vào việc có công ty nhận hay không, nên hãy nộp CV sớm (từ tuần 8-9).
- Dùng AI để giải thích và review, nhưng tự gõ phần logic cốt lõi và giải thích được từng dòng.